

# Документ вимог до проєкту: SmartLib (Система управління бібліотекою)

## 1. Зацікавлені сторони (Stakeholders)

- **Бібліотекарі (Операційні користувачі):** Працівники бібліотеки, які безпосередньо взаємодіють із системою щодня: реєструють видачу й повернення книг, оформлюють нових читачів та керують каталогом.
- **Адміністратори книжкового фонду (Management):** Керівники або старші бібліографи, які аналізують статистику, формують аналітичні звіти та контролюють стан книжкового фонду.
- **Читачі (Кінцеві споживачі):** Студенти, викладачі та інші відвідувачі бібліотеки, які шукають літературу та перевіряють її наявність.
- **Команда розробників (TeamGirls):**
  - Шелеп Софія (**Data & Architecture**)
  - Мандзювата Оксана (**Frontend / Desktop UI**)
  - Назарова Юлія (**Core Backend**)

## 2. Функціональні вимоги (Functional Requirements)

1. **Каталог книг:** Система повинна забезпечувати пошук книг за різними параметрами (автор, жанр, рік видання).
2. **Управління фондом (CRUD):** Адміністратори та бібліотекарі повинні мати можливість додавати нові книги, редагувати наявну інформацію та здійснювати списання застарілих чи пошкоджених примірників.
3. **Облік видачі та повернення:** Реєстрація процесу видачі книги читачу із фіксацією дати видачі й повернення, а також оновлення статусу книги в реальному часі.
4. **Формування звітів:** Генерація аналітичних звітів про роботу бібліотеки (наприклад, найпопулярніші книги, активні читачі, книги на руках).
5. **Інтерфейс користувача (Desktop UI):** Графічний інтерфейс для робочого місця бібліотекаря з відображенням даних у таблицях (DataGrid) та динамічним оновленням станів за допомогою подій і делегатів.

### 3. Нефункціональні вимоги (Non-Functional Requirements)

- **Технічний стек:** Застосунок повинен бути розроблений для десктопних операційних систем сімейства Windows. Розробка на платформі .NET (C#) з використанням WPF або Windows Forms для графічного інтерфейсу. Для збереження даних повинна використовуватися реляційна система управління базами даних PostgreSQL.
- **Продуктивність:** Час обробки пошукового запиту в каталозі (до 100 000 записів) та відображення результатів на екрані не повинен перевищувати 2 секунд.
- **Надійність:** Система повинна гарантувати збереження цілісності даних про видачу та повернення книг у разі раптового збою живлення або критичної помилки операційної системи.
- **Зручність використання:** Інтерфейс повинен бути спроектований так, щоб новий співробітник міг виконати базові операції (оформлення видачі та повернення книги) не більше ніж за 4 кліки мишею після вибору потрібного читача. Час на самостійне навчання базової роботи з програмою не повинен перевищувати 1 годину.
- **Супроводжуваність:** Архітектура системи повинна забезпечувати повне відокремлення логіки доступу до даних від графічного інтерфейсу користувача, що дозволить оновлювати UI без внесення змін до бізнес-логіки бази даних.
- **Безпека:** Система повинна обмежувати доступ до функцій списання книг та редагування профілів читачів, надаючи його виключно користувачам із правами "Адміністратор".

### 4. Історії користувача (User Stories) та критерії приймання (Acceptance Criteria)

#### Історія 1: Пошук книги в каталозі

- **As a (Як):** бібліотекар або читач,
- **I want (Хочу):** швидко знаходити книги за автором, жанром або роком видання через рядок пошуку,
- **So that (Щоб):** зекономити час на пошук потрібної літератури та перевірити її наявність.
- **Критерії приймання (Acceptance Criteria):**
  1. У вікні застосунку є поле вводу для пошуку та кнопка пошуку.

2. При введенні критерію та натисканні кнопки таблиця (DataGrid) оновлюється й показує лише відповідні результати.
3. Якщо нічого не знайдено, система виводить відповідне сповіщення.

- **Критерії завершеності (Definition of Done):**

1. Інтерфейс (XAML/Forms) зверстано згідно з макетом (присутні поле вводу та кнопка).
2. Логіка пошуку (LINQ-запити до колекції або SQL-запити до БД) успішно відпрацьовує без зависання інтерфейсу (використовується асинхронність, якщо потрібно).
3. Написано автоматизовані тести, які перевіряють, що пошук коректно фільтрує дані за автором, жанром та роком.
4. Код пройшов перевірку (Code Review) іншим учасником команди (наприклад, Юлією або Софією).
5. Відсутні помилки компіляції, зміни злиті в основну гілку репозиторію на GitHub.

## Історія 2: Облік видачі книги

- **As a (Як):** бібліотекар,
- **I want (Хочу):** зареєструвати видачу книги конкретному читачу в один клік,
- **So that (Щоб):** статус книги автоматично змінився на "Видана", а інформація збереглася в базі даних.
- **Критерії приймання (Acceptance Criteria):**
  1. У інтерфейсі доступна дія вибору книги та читача для оформлення видачі.

2. Після підтвердження видачі статус книги в таблиці змінюється миттєво (завдяки подіям та делегатам).
3. Запис про позику (Loan) створюється в базі даних PostgreSQL

- **Критерії завершеності (Definition of Done):**

1. Створено зв'язок між таблицями Books, Readers та Loans у PostgreSQL за допомогою зовнішніх ключів (Foreign Keys).
2. Реалізовано архітектурний патерн з використанням подій (Events) та делегатів (Delegates) у C# для миттєвого оновлення DataGridView без перезавантаження всього вікна.
3. Оброблено винятки (Exceptions): система не дозволяє видати книгу, яка вже має статус "Видана" або списана.
4. Написано тести на створення запису в БД та зміну статусу об'єкта книги.
5. Функціонал перевірено, код відповідає стандартам проєкту та злитий в основну гілку.

### Історія 3: Перегляд списку боржників

- **As a (Як):** адміністратор.
- **I want (Хочу):** бачити список усіх читачів, які протермінували повернення книг.
- **So that (Щоб):** мати змогу зв'язатися з ними.
- **Критерії приймання (Acceptance Criteria):**
  1. В інтерфейсі доступна окрема кнопка або розділ "Боржники".
  2. При переході в розділ генерується список видач, де очікувана дата повернення є меншою за поточну дату, а книга ще не повернута.
  3. У згенерованому списку відображається ПІБ читача, назва книги та кількість днів протермінування.
- **Критерії завершеності (Definition of Done):**
  1. SQL-запит до PostgreSQL коректно вираховує різницю між поточною датою (CURRENT\_DATE) та очікуваною датою повернення для обчислення днів протермінування.
  2. У UI (DataGridView) налаштовано візуальне виділення (наприклад, червоним кольором) для поля "кількість днів протермінування".

3. Проведено тестування з тестовими даними (Mock data), де змодельовані ситуації з простроченням на 1, 10 та 30 днів.
4. Перевірено безпеку: доступ до цього розділу та списку боржників має лише користувач із роллю "Адміністратор".
5. Код задокументований, перевірений (Code Review) та доданий до головної гілки.